Kubernetes 入門

aoki-taquan

2026 年 03 月 15 日

目次

1 はじめに 6
1.1 Kubernetes とは 6
1.2 なぜ Kubernetes が必要なのか 6
1.3 Kubernetes の主な特徴 6
1.3.1 自動修復（Self-healing） 6
1.3.2 水平スケーリング 6
1.3.3 サービスディスカバリとロードバランシング 6
1.3.4 自動ロールアウトとロールバック 6
1.3.5 宣言的設定 7
1.4 このガイドについて 7
1.5 対象読者 7
1.6 前提知識 7
2 環境構築 8
2.1 ローカル開発環境の選択肢 8
2.2 Minikube のインストール 8
2.2.1 macOS 8
2.2.2 Linux 8
2.2.3 Windows 8
2.3 kubectl のインストール 8
2.3.1 macOS 9
2.3.2 Linux 9
2.3.3 Windows 9
2.3.4 インストール確認 9
2.4 Minikube クラスタの起動 9
2.4.1 クラスタの起動 9
2.4.2 クラスタの状態確認 9
2.4.3 クラスタ情報の確認 10
2.5 kubectl の基本操作 10
2.5.1 ノードの確認 10
2.5.2 全リソースの確認 10
2.5.3 名前空間の確認 10
2.6 Minikube の管理コマンド 10
2.7 Minikube ダッシュボード 10
3 基本概念 12
3.1 Kubernetes のアーキテクチャ 12
3.1.1 コントロールプレーン 12
3.1.2 ワーカーノード 12
3.2 主要なリソース 12
3.2.1 ワークロード系リソース 12

3.2.2 ネットワーク系リソース 12
3.2.3 設定・ストレージ系リソース 13
3.2.4 クラスタ管理系リソース 13
3.3 マニフェストファイル 13
3.3.1 apiVersion 13
3.4 ラベルとセレクタ 13
3.4.1 ラベル 13
3.4.2 セレクタ 14
3.5 kubectl の基本コマンド 14
3.5.1 出力フォーマット 14
4 Pod 15
4.1 Pod とは 15
4.2 Pod の作成 15
4.2.1 マニフェストで作成 15
4.2.2 コマンドで直接作成 15
4.3 Pod の状態確認 16
4.3.1 一覧表示 16
4.3.2 詳細情報 16
4.4 Pod のライフサイクル 16
4.5 Pod 内でのコマンド実行 16
4.6 Pod のログ確認 17
4.7 ポートフォワーディング 17
4.8 マルチコンテナ Pod 17
4.9 リソース制限 17
4.10 ヘルスチェック 18
4.10.1 livenessProbe 18
4.10.2 readinessProbe 18
4.10.3 startupProbe 18
4.11 Pod の削除 19
5 Deployment 20
5.1 Deployment とは 20
5.2 Deployment と ReplicaSet の関係 20
5.3 Deployment の作成 20
5.4 状態確認 21
5.5 スケーリング 21
5.5.1 コマンドでスケーリング 21
5.5.2 マニフェストでスケーリング 22
5.6 ローリングアップデート 22
5.6.1 イメージの更新 22
5.6.2 アップデートの進行状況を確認 22
5.6.3 アップデート戦略の設定 22
5.7 ロールバック 23

5.7.1 ロールアウト履歴の確認 23
5.7.2 特定リビジョンの詳細を確認 23
5.7.3 前のバージョンに戻す 23
5.7.4 特定のリビジョンに戻す 23
5.8 一時停止と再開 23
5.9 Deployment の削除 23
6 Service 25
6.1 Service とは 25
6.2 Service の種類 25
6.3 ClusterIP Service 25
6.3.1 確認 26
6.3.2 クラスタ内からのアクセス 26
6.4 NodePort Service 26
6.4.1 Minikube でのアクセス 26
6.5 LoadBalancer Service 26
6.6 DNS によるサービスディスカバリ 27
6.7 複数ポートの公開 27
6.8 Headless Service 28
6.9 Service の確認と削除 28
7 ConfigMap と Secret 29
7.1 設定管理の必要性 29
7.2 ConfigMap 29
7.2.1 ConfigMap の作成 29
7.2.2 ConfigMap の確認 29
7.2.3 ConfigMap の利用方法 30
7.3 Secret 31
7.3.1 Secret の種類 31
7.3.2 Secret の作成 31
7.3.3 Secret の利用方法 32
7.4 Secret の注意点 32
8 ストレージ 34
8.1 コンテナのストレージの課題 34
8.2 Volume 34
8.2.1 emptyDir 34
8.2.2 hostPath 34
8.3 PersistentVolume（PV）と PersistentVolumeClaim（PVC） 35
8.3.1 PV の作成 35
8.3.2 アクセスモード 35
8.3.3 再利用ポリシー 35
8.3.4 PVC の作成 36
8.3.5 PVC を Pod で使用 36
8.4 StorageClass 36

8.5 ストレージの確認 37
9 Namespace 38
9.1 Namespace とは 38
9.2 デフォルトの Namespace 38
9.3 Namespace の操作 38
9.3.1 作成 38
9.3.2 一覧表示 38
9.3.3 特定の Namespace 内のリソースを表示 38
9.3.4 デフォルト Namespace の変更 39
9.4 Namespace にリソースを作成 39
9.5 ResourceQuota 39
9.5.1 ResourceQuota の確認 40
9.6 LimitRange 40
9.7 Namespace の削除 41
10 実践運用 42
10.1 アプリケーションのデプロイ例 42
10.1.1 Deployment の作成 42
10.1.2 Service の作成 43
10.1.3 ConfigMap の作成 43
10.1.4 一括適用 43
10.2 トラブルシューティング 43
10.2.1 Pod が起動しない場合 43
10.2.2 よくある Pod のエラー 44
10.2.3 Service に接続できない場合 44
10.3 リソースの監視 44
10.3.1 リソース使用量の確認 44
10.4 ラベルを活用した運用 45
10.4.1 ラベルの追加・変更 45
10.4.2 ラベルによるフィルタリング 45
10.5 便利な kubectl コマンド 45
10.6 次のステップ 45
10.6.1 参考リソース 46

1 はじめに

1.1 Kubernetes とは

Kubernetes（クバネティス、略称: K8s）は、コンテナ化されたアプリケーションのデプロイ、スケーリング、管理を自動化するためのオープンソースプラットフォームです。Google が社内で使用していたコンテナ管理システム「Borg」の経験をもとに開発され、2014 年にオープンソースとして公開されました。現在は Cloud Native Computing Foundation（CNCF）によって管理されています。

Kubernetes という名前はギリシャ語で「操舵手」や「パイロット」を意味します。K8s という略称は、「K」と「s」の間に 8 文字あることに由来しています。

1.2 なぜ Kubernetes が必要なのか

コンテナ技術（Docker 等）の普及により、アプリケーションのパッケージングと実行が容易になりました。しかし、本番環境で多数のコンテナを運用するには、以下のような課題があります。

- **スケーリング**: トラフィックの増減に応じてコンテナ数を調整する必要がある
- **障害対応**: コンテナやノードの障害時に自動的に復旧させたい
- **ロードバランシング**: 複数のコンテナにトラフィックを分散させたい
- **ローリングアップデート**: ダウンタイムなしでアプリケーションを更新したい
- **設定管理**: 環境ごとに異なる設定を安全に管理したい

Kubernetes はこれらの課題を解決し、コンテナのオーケストレーション（統合管理）を実現します。

1.3 Kubernetes の主な特徴

1.3.1 自動修復（Self-healing）

コンテナが異常終了した場合、Kubernetes は自動的にコンテナを再起動します。ノードに障害が発生した場合は、別のノードでコンテナを再スケジュールします。

1.3.2 水平スケーリング

CPU 使用率やカスタムメトリクスに基づいて、アプリケーションのレプリカ数を自動的に増減できます。

1.3.3 サービスディスカバリとロードバランシング

Kubernetes は DNS 名や IP アドレスを使ってコンテナを公開できます。トラフィックが多い場合は自動的に負荷分散を行います。

1.3.4 自動ロールアウトとロールバック

アプリケーションの更新時に段階的に新しいバージョンをデプロイし、問題が発生した場合は自動的にロールバックできます。

1.3.5 宣言的設定

YAML や JSON のマニフェストファイルで望ましい状態を宣言し、Kubernetes がその状態を自動的に維持します。

1.4 このガイドについて

このガイドでは、Kubernetes の基本概念から実践的な使い方まで、初心者向けにステップバイステップで解説します。

1.5 対象読者

- コンテナ（Docker）の基本的な知識がある方
- Kubernetes をこれから学び始めたい方
- Linux コマンドの基本操作ができる方

1.6 前提知識

以下の知識があると、本ガイドの内容をより理解しやすくなります。

- Docker の基本操作（イメージのビルド、コンテナの起動・停止）
- YAML 記法の基礎
- Linux/macOS のターミナル操作
- ネットワークの基礎知識（IP アドレス、ポート、DNS）

2 環境構築

2.1 ローカル開発環境の選択肢

Kubernetes をローカル環境で試すためのツールがいくつかあります。

ツール	特徴	リソース消費	推奨用途
Minikube	最も一般的、豊富な機能	中	学習・開発
kind	Docker 上で動作、軽量	小	CI/CD・テスト
Docker Desktop	GUI 付き、簡単セットアップ	大	初心者向け
k3d	k3s を Docker 上で実行	小	軽量環境

本ガイドでは、最も広く使われている Minikube を使用します。

2.2 Minikube のインストール

2.2.1 macOS

Homebrew を使ってインストールします。

```
brew install minikube
```

2.2.2 Linux

バイナリを直接ダウンロードしてインストールします。

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
rm minikube-linux-amd64
```

2.2.3 Windows

WinGet を使ってインストールします。

```
winget install Kubernetes.minikube
```

2.3 kubectl のインストール

kubectl は Kubernetes クラスタを操作するためのコマンドラインツールです。

2.3.1 macOS

```
brew install kubectl
```

2.3.2 Linux

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/
bin/linux/amd64/kubectl"
sudo install kubectl /usr/local/bin/kubectl
rm kubectl
```

2.3.3 Windows

```
winget install Kubernetes.kubectl
```

2.3.4 インストール確認

```
kubectl version --client
```

2.4 Minikube クラスタの起動

2.4.1 クラスタの起動

```
minikube start
```

デフォルトでは Docker ドライバを使用してシングルノードの Kubernetes クラスタが作成されます。

リソースをカスタマイズする場合は以下のように指定します。

```
minikube start --cpus=4 --memory=8192 --driver=docker
```

2.4.2 クラスタの状態確認

```
minikube status
```

以下のような出力が表示されれば正常です。

```
minikube
type: Control Plane
host: Running
kubelet: Running
```

```
apiserver: Running
kubeconfig: Configured
```

2.4.3 クラスタ情報の確認

```
kubectl cluster-info
```

2.5 kubectl の基本操作

2.5.1 ノードの確認

```
kubectl get nodes
```

```
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   1m    v1.31.0
```

2.5.2 全リソースの確認

```
kubectl get all
```

2.5.3 名前空間の確認

```
kubectl get namespaces
```

2.6 Minikube の管理コマンド

コマンド	説明
`minikube start`	クラスタを起動する
`minikube stop`	クラスタを停止する（データは保持）
`minikube delete`	クラスタを削除する
`minikube dashboard`	Web UI ダッシュボードを開く
`minikube ssh`	Minikube ノードに ssh 接続する
`minikube tunnel`	LoadBalancer サービスにアクセス可能にする
`minikube addons list`	利用可能なアドオンを一覧表示する

2.7 Minikube ダッシュボード

Kubernetes には Web ベースのダッシュボードがあります。以下のコマンドで起動できます。

```
minikube dashboard
```

ダッシュボードではクラスタの状態を GUI で確認・管理できます。学習中は CLI（kubectl）とダッシュボードを併用すると理解が深まります。

3 基本概念

3.1 Kubernetes のアーキテクチャ

Kubernetes クラスタは、大きく分けて**コントロールプレーン**と**ワーカーノード**の 2 つの要素で構成されます。

3.1.1 コントロールプレーン

クラスタ全体を管理する頭脳の役割を担います。

コンポーネント	役割
kube-apiserver	Kubernetes API を提供するフロントエンド。kubectl からのリクエストを受け付ける
etcd	クラスタの全データを保存する分散キーバリューストア
kube-scheduler	新しい Pod をどのノードに配置するか決定する
kube-controller-manager	各種コントローラー（ReplicaSet、Deployment など）を実行する

3.1.2 ワーカーノード

実際にコンテナが動作するマシンです。

コンポーネント	役割
kubelet	ノード上で Pod の起動・管理を行うエージェント
kube-proxy	ネットワークルールを管理し、Service への通信を転送する
コンテナランタイム	コンテナを実行する（containerd、CRI-O など）

3.2 主要なリソース

Kubernetes では、すべてのものが「リソース」として管理されます。以下が主要なリソースです。

3.2.1 ワークロード系リソース

- **Pod**: コンテナの最小デプロイ単位
- **ReplicaSet**: Pod のレプリカ数を維持する
- **Deployment**: ReplicaSet を管理し、ローリングアップデートを実現する
- **DaemonSet**: 各ノードに Pod を 1 つずつ配置する
- **StatefulSet**: ステートフルなアプリケーション向けの Pod 管理
- **Job / CronJob**: 一回限り、または定期的なバッチ処理を実行する

3.2.2 ネットワーク系リソース

- **Service**: Pod へのネットワークアクセスを提供する

- **Ingress**: HTTP/HTTPS のルーティングを管理する

3.2.3 設定・ストレージ系リソース

- **ConfigMap**: 設定データを管理する
- **Secret**: 機密情報を管理する
- **PersistentVolume (PV)**: 永続ストレージを提供する
- **PersistentVolumeClaim (PVC)**: PV の利用を要求する

3.2.4 クラスタ管理系リソース

- **Namespace**: クラスタ内のリソースを論理的に分離する
- **ServiceAccount**: Pod 用の認証情報を管理する
- **Role / ClusterRole**: アクセス権限を定義する

3.3 マニフェストファイル

Kubernetes ではリソースの定義を YAML ファイル（マニフェスト）で記述します。すべてのマニフェストには以下の共通フィールドがあります。

```yaml
apiVersion: v1          # APIのバージョン
kind: Pod               # リソースの種類
metadata:               # メタデータ
  name: my-pod          # リソース名
  labels:               # ラベル（任意のキー・バリュー）
    app: my-app
spec:                   # リソースの仕様（種類によって異なる）
  ...
```

3.3.1 apiVersion

リソースの種類によって使用する API バージョンが異なります。

リソース	apiVersion
Pod, Service, ConfigMap, Secret	`v1`
Deployment, ReplicaSet	`apps/v1`
Ingress	`networking.k8s.io/v1`
Job, CronJob	`batch/v1`

3.4 ラベルとセレクタ

3.4.1 ラベル

ラベルはリソースに付与するキー・バリューのペアです。リソースの分類やフィルタリングに使用します。

```yaml
metadata:
  labels:
    app: web-server
    env: production
    tier: frontend
```

3.4.2 セレクタ

セレクタはラベルを使ってリソースを選択する仕組みです。Deployment が Pod を管理したり、Service が Pod にトラフィックを送る際に使われます。

```yaml
selector:
  matchLabels:
    app: web-server
```

3.5 kubectl の基本コマンド

コマンド	説明
`kubectl get <リソース>`	リソースの一覧を表示する
`kubectl describe <リソース> <名前>`	リソースの詳細を表示する
`kubectl apply -f <ファイル>`	マニフェストを適用する（作成・更新）
`kubectl delete -f <ファイル>`	マニフェストのリソースを削除する
`kubectl logs <Pod名>`	Pod のログを表示する
`kubectl exec -it <Pod名> -- <コマンド>`	Pod 内でコマンドを実行する
`kubectl port-forward <Pod名> <ローカルポート>:<Podポート>`	Pod へポートフォワーディングする

3.5.1 出力フォーマット

`-o` オプションで出力形式を変更できます。

```bash
kubectl get pods -o wide          # 詳細情報を表示
kubectl get pods -o yaml          # YAML形式で出力
kubectl get pods -o json          # JSON形式で出力
kubectl get pods -o name          # リソース名のみ表示
```

4 Pod

4.1 Pod とは

Pod は Kubernetes におけるデプロイの最小単位です。1 つ以上のコンテナをまとめたグループであり、同一 Pod 内のコンテナはネットワークとストレージを共有します。

多くの場合、1 つの Pod には 1 つのコンテナを配置しますが、密接に連携する複数のコンテナを同一 Pod に配置するパターン（サイドカーパターン）もあります。

4.2 Pod の作成

4.2.1 マニフェストで作成

`nginx-pod.yaml` というファイルを作成します。

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80
```

マニフェストを適用して Pod を作成します。

```bash
kubectl apply -f nginx-pod.yaml
```

4.2.2 コマンドで直接作成

簡単なテスト用途では `kubectl run` コマンドでも作成できます。

```bash
kubectl run nginx-pod --image=nginx:1.27 --port=80
```

4.3 Pod の状態確認

4.3.1 一覧表示

```
kubectl get pods
```

```
NAME        READY   STATUS    RESTARTS   AGE
nginx-pod   1/1     Running   0          30s
```

4.3.2 詳細情報

```
kubectl describe pod nginx-pod
```

Pod の詳細情報が表示されます。トラブルシューティングの際に特に重要な情報は以下の通りです。

- **Events**: Pod に関するイベント（スケジュール、イメージのプル、起動など）
- **Conditions**: Pod の状態条件
- **Containers**: 各コンテナの状態

4.4 Pod のライフサイクル

Pod は以下の状態（Phase）を遷移します。

状態	説明
Pending	Pod が作成されたがまだコンテナが起動していない
Running	少なくとも 1 つのコンテナが実行中
Succeeded	すべてのコンテナが正常に終了した
Failed	少なくとも 1 つのコンテナが異常終了した
Unknown	Pod の状態を取得できない

4.5 Pod 内でのコマンド実行

```
# Podのシェルに接続
kubectl exec -it nginx-pod -- /bin/bash

# 単一コマンドの実行
kubectl exec nginx-pod -- cat /etc/nginx/nginx.conf
```

4.6 Pod のログ確認

```
# ログを表示
kubectl logs nginx-pod

# リアルタイムでログを追跡
kubectl logs -f nginx-pod

# 直近の100行を表示
kubectl logs --tail=100 nginx-pod
```

4.7 ポートフォワーディング

Pod に直接アクセスするにはポートフォワーディングを使用します。

```
kubectl port-forward nginx-pod 8080:80
```

ブラウザで `http://localhost:8080` にアクセスすると nginx の画面が表示されます。

4.8 マルチコンテナ Pod

1 つの Pod に複数のコンテナを配置する場合の例です。

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: multi-container-pod
spec:
  containers:
    - name: app
      image: nginx:1.27
      ports:
        - containerPort: 80
    - name: sidecar
      image: busybox:1.36
      command: ["sh", "-c", "while true; do echo sidecar running; sleep 30; done"]
```

マルチコンテナ Pod ではコンテナ名を指定してログを確認します。

```
kubectl logs multi-container-pod -c sidecar
```

4.9 リソース制限

コンテナに対して CPU とメモリの制限を設定できます。

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: resource-limited-pod
spec:
  containers:
    - name: app
      image: nginx:1.27
      resources:
        requests:          # 最低限確保するリソース
          memory: "64Mi"
          cpu: "250m"
        limits:            # 使用上限
          memory: "128Mi"
          cpu: "500m"
```

- **requests**: スケジューラがノードを選択する際の基準。この量のリソースが利用可能なノードに配置される
- **limits**: コンテナが使用できるリソースの上限。超過すると OOMKill（メモリ）やスロットリング（CPU）が発生する

4.10 ヘルスチェック

Kubernetes は 3 種類のヘルスチェック（Probe）をサポートしています。

4.10.1 livenessProbe

コンテナが正常に動作しているか確認します。失敗するとコンテナが再起動されます。

4.10.2 readinessProbe

コンテナがリクエストを受け付けられる状態か確認します。失敗すると Service のエンドポイントから除外されます。

4.10.3 startupProbe

コンテナの起動が完了したか確認します。起動が遅いアプリケーションに使用します。

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: health-check-pod
spec:
  containers:
    - name: app
      image: nginx:1.27
      livenessProbe:
        httpGet:
```

```yaml
          path: /
          port: 80
        initialDelaySeconds: 5
        periodSeconds: 10
      readinessProbe:
        httpGet:
          path: /
          port: 80
        initialDelaySeconds: 3
        periodSeconds: 5
```

4.11 Pod の削除

```bash
# 名前を指定して削除
kubectl delete pod nginx-pod

# マニフェストで削除
kubectl delete -f nginx-pod.yaml
```

5 Deployment

5.1 Deployment とは

Deployment は、Pod のデプロイと管理を行うリソースです。Pod を直接管理するのではなく、ReplicaSet を通じて Pod を管理します。Deployment を使うことで、以下の機能が利用できます。

- Pod のレプリカ数の維持
- ローリングアップデート
- ロールバック
- スケーリング

実運用では Pod を直接作成せず、Deployment を使うのが一般的です。

5.2 Deployment と ReplicaSet の関係

Deployment の管理構造は以下のようになっています。

```
Deployment
  └── ReplicaSet
        ├── Pod
        ├── Pod
        └── Pod
```

- **Deployment**: 望ましい状態を宣言し、ReplicaSet を管理する
- **ReplicaSet**: 指定された数の Pod レプリカを維持する
- **Pod**: 実際にコンテナが動作する単位

5.3 Deployment の作成

`nginx-deployment.yaml` を作成します。

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
```

```yaml
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

マニフェストの主要フィールドを解説します。

- `spec.replicas`: 維持する Pod の数
- `spec.selector`: 管理対象の Pod をラベルで選択
- `spec.template`: 作成する Pod のテンプレート（ラベルは selector と一致させる）

```yaml
kubectl apply -f nginx-deployment.yaml
```

5.4 状態確認

```yaml
# Deploymentの確認
kubectl get deployments

# ReplicaSetの確認
kubectl get replicasets

# Podの確認
kubectl get pods
```

```
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment   3/3     3            3           30s
```

- **READY**: 準備完了の Pod 数 / 期待する Pod 数
- **UP-TO-DATE**: 最新の Pod テンプレートで作成された Pod 数
- **AVAILABLE**: 利用可能な Pod 数

5.5 スケーリング

5.5.1 コマンドでスケーリング

```yaml
# レプリカ数を5に変更
kubectl scale deployment nginx-deployment --replicas=5

# 確認
kubectl get pods
```

5.5.2 マニフェストでスケーリング

`spec.replicas` の値を変更して `kubectl apply` を実行します。

```yaml
spec:
  replicas: 5
```

```
kubectl apply -f nginx-deployment.yaml
```

5.6 ローリングアップデート

Deployment の最大の利点の 1 つが、ダウンタイムなしのローリングアップデートです。

5.6.1 イメージの更新

```
# コマンドでイメージを更新
kubectl set image deployment/nginx-deployment nginx=nginx:1.27.3
```

またはマニフェストのイメージタグを変更して `kubectl apply` を実行します。

5.6.2 アップデートの進行状況を確認

```
kubectl rollout status deployment/nginx-deployment
```

```
Waiting for deployment "nginx-deployment" rollout to finish: 1 out of 3 new replicas
have been updated...
Waiting for deployment "nginx-deployment" rollout to finish: 2 out of 3 new replicas
have been updated...
deployment "nginx-deployment" successfully rolled out
```

5.6.3 アップデート戦略の設定

```yaml
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1           # 同時に追加できるPod数
      maxUnavailable: 0     # 同時に停止できるPod数
```

- `maxSurge`：アップデート中に追加で作成できる Pod 数（またはパーセント）
- `maxUnavailable`：アップデート中に利用不可にできる Pod 数（またはパーセント）

5.7 ロールバック

5.7.1 ロールアウト履歴の確認

```
kubectl rollout history deployment/nginx-deployment
```

```
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
```

5.7.2 特定リビジョンの詳細を確認

```
kubectl rollout history deployment/nginx-deployment --revision=1
```

5.7.3 前のバージョンに戻す

```
kubectl rollout undo deployment/nginx-deployment
```

5.7.4 特定のリビジョンに戻す

```
kubectl rollout undo deployment/nginx-deployment --to-revision=1
```

5.8 一時停止と再開

大きな変更を複数回に分けて行いたい場合、ロールアウトを一時停止できます。

```
# ロールアウトを一時停止
kubectl rollout pause deployment/nginx-deployment

# 複数の変更を適用
kubectl set image deployment/nginx-deployment nginx=nginx:1.27.3
kubectl set resources deployment/nginx-deployment -c=nginx --
limits=cpu=200m,memory=512Mi

# ロールアウトを再開（一度にデプロイされる）
kubectl rollout resume deployment/nginx-deployment
```

5.9 Deployment の削除

```
# 名前で削除
kubectl delete deployment nginx-deployment
```

```
# マニフェストで削除
kubectl delete -f nginx-deployment.yaml
```

Deployment を削除すると、管理下の ReplicaSet と Pod も自動的に削除されます。

6 Service

6.1 Service とは

Service は、Pod への安定したネットワークアクセスを提供するリソースです。Pod は起動するたびに IP アドレスが変わりますが、Service を使うことで固定のエンドポイントを通じて Pod にアクセスできます。

Service はラベルセレクタを使って対象の Pod を動的に選択し、トラフィックをロードバランシングします。

6.2 Service の種類

種類	説明	用途
ClusterIP	クラスタ内部のみでアクセス可能な仮想 IP を割り当てる	内部通信
NodePort	各ノードのポートを通じて外部からアクセス可能にする	開発・テスト
LoadBalancer	外部ロードバランサーを使って公開する	本番環境
ExternalName	外部の DNS 名にマッピングする	外部サービス連携

6.3 ClusterIP Service

クラスタ内部でのみアクセスできる Service です。デフォルトの Service 種類です。

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: ClusterIP
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80          # Serviceのポート
      targetPort: 80    # Podのポート
```

```
kubectl apply -f nginx-service.yaml
```

6.3.1 確認

```
kubectl get services
```

```
NAME            TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
nginx-service   ClusterIP   10.96.42.123   <none>        80/TCP    30s
```

6.3.2 クラスタ内からのアクセス

クラスタ内の Pod から Service にアクセスするには、Service 名をホスト名として使用します。

```
# 一時的なPodからcurlでアクセス
kubectl run curl-test --image=curlimages/curl --rm -it -- curl http://nginx-service
```

6.4 NodePort Service

各ノードの指定されたポートを通じて外部からアクセスできる Service です。

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30080    # ノードのポート（30000-32767）
```

NodePort は 30000〜32767 の範囲で指定できます。省略するとランダムに割り当てられます。

6.4.1 Minikube でのアクセス

```
minikube service nginx-nodeport --url
```

6.5 LoadBalancer Service

クラウド環境（AWS、GCP、Azure など）で外部ロードバランサーを自動的にプロビジョニングする Service です。

```yaml
apiVersion: v1
kind: Service
```

```yaml
metadata:
  name: nginx-loadbalancer
spec:
  type: LoadBalancer
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

Minikube で使用する場合は、別のターミナルで `minikube tunnel` を実行する必要があります。

```
# 別のターミナルで実行
minikube tunnel
```

6.6 DNS によるサービスディスカバリ

Kubernetes クラスタ内では DNS が自動的に設定されます。以下の形式で Service にアクセスできます。

```
<Service名>.<Namespace>.svc.cluster.local
```

同じ Namespace 内であれば Service 名だけでアクセスできます。

```
# 同じNamespace内
curl http://nginx-service

# 別のNamespaceのServiceにアクセス
curl http://nginx-service.default.svc.cluster.local
```

6.7 複数ポートの公開

1 つの Service で複数のポートを公開できます。

```yaml
apiVersion: v1
kind: Service
metadata:
  name: multi-port-service
spec:
  selector:
    app: my-app
  ports:
    - name: http
      protocol: TCP
```

```
      port: 80
      targetPort: 8080
    - name: https
      protocol: TCP
      port: 443
      targetPort: 8443
```

複数ポートを定義する場合は、各ポートに `name` を指定する必要があります。

6.8 Headless Service

ClusterIP を割り当てず、DNS で Pod の IP アドレスを直接返す Service です。StatefulSet と組み合わせて使用されます。

```
apiVersion: v1
kind: Service
metadata:
  name: headless-service
spec:
  clusterIP: None
  selector:
    app: my-app
  ports:
    - port: 80
      targetPort: 80
```

6.9 Service の確認と削除

```
# Serviceの一覧
kubectl get services

# Serviceの詳細（エンドポイント情報を含む）
kubectl describe service nginx-service

# エンドポイントの確認
kubectl get endpoints nginx-service

# 削除
kubectl delete service nginx-service
```

7 ConfigMap と Secret

7.1 設定管理の必要性

アプリケーションの設定をコンテナイメージに直接埋め込むと、環境ごとにイメージを作り直す必要があります。Kubernetes では ConfigMap と Secret を使って設定をコンテナから分離し、柔軟に管理できます。

7.2 ConfigMap

ConfigMap は、機密性のない設定データをキー・バリューのペアで保存するリソースです。

7.2.1 ConfigMap の作成

7.2.1.1 マニフェストで作成

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: "production"
  APP_DEBUG: "false"
  APP_LOG_LEVEL: "info"
```

```bash
kubectl apply -f app-config.yaml
```

7.2.1.2 コマンドで作成

```bash
# リテラル値から作成
kubectl create configmap app-config \
  --from-literal=APP_ENV=production \
  --from-literal=APP_DEBUG=false

# ファイルから作成
kubectl create configmap nginx-config --from-file=nginx.conf

# ディレクトリから作成
kubectl create configmap configs --from-file=./config-dir/
```

7.2.2 ConfigMap の確認

```bash
kubectl get configmaps
kubectl describe configmap app-config
```

7.2.3 ConfigMap の利用方法

7.2.3.1 環境変数として利用

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
    - name: app
      image: my-app:1.0
      envFrom:
        - configMapRef:
            name: app-config
```

`envFrom` を使うと、ConfigMap の全エントリが環境変数として設定されます。

特定のキーだけを使う場合は以下のように記述します。

```yaml
env:
  - name: APP_ENVIRONMENT
    valueFrom:
      configMapKeyRef:
        name: app-config
        key: APP_ENV
```

7.2.3.2 ボリュームとしてマウント

設定ファイルをそのままマウントしたい場合に使用します。

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
    - name: app
      image: nginx:1.27
      volumeMounts:
        - name: config-volume
          mountPath: /etc/nginx/conf.d
  volumes:
    - name: config-volume
      configMap:
        name: nginx-config
```

7.3 Secret

Secret は、パスワード、API キー、証明書などの機密情報を保存するリソースです。Base64 でエンコードされて保存されます。

7.3.1 Secret の種類

種類	用途
Opaque	任意のデータ（デフォルト）
kubernetes.io/dockerconfigjson	Docker レジストリの認証情報
kubernetes.io/tls	TLS 証明書と秘密鍵
kubernetes.io/basic-auth	Basic 認証の資格情報

7.3.2 Secret の作成

7.3.2.1 マニフェストで作成

値は Base64 でエンコードする必要があります。

```
echo -n 'mypassword' | base64
# bXlwYXNzd29yZA==
```

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
data:
  DB_PASSWORD: bXlwYXNzd29yZA==
  API_KEY: c2VjcmV0LWtleQ==
```

`stringData` フィールドを使うと Base64 エンコードなしで記述できます。

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
stringData:
  DB_PASSWORD: "mypassword"
  API_KEY: "secret-key"
```

7.3.2.2 コマンドで作成

```
kubectl create secret generic app-secret \
  --from-literal=DB_PASSWORD=mypassword \
  --from-literal=API_KEY=secret-key
```

7.3.3 Secret の利用方法

7.3.3.1 環境変数として利用

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
    - name: app
      image: my-app:1.0
      env:
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: app-secret
              key: DB_PASSWORD
```

7.3.3.2 ボリュームとしてマウント

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
    - name: app
      image: my-app:1.0
      volumeMounts:
        - name: secret-volume
          mountPath: /etc/secrets
          readOnly: true
  volumes:
    - name: secret-volume
      secret:
        secretName: app-secret
```

7.4 Secret の注意点

- Secret の Base64 エンコードは暗号化ではありません。誰でもデコードできます
- Secret をマニフェストに含める場合、Git リポジトリにコミットしないよう注意してください

- 本番環境では External Secrets や Sealed Secrets などのツールの導入を検討してください
- etcd の暗号化を有効にすることで、保存時のセキュリティを強化できます

8 ストレージ

8.1 コンテナのストレージの課題

コンテナのファイルシステムは一時的なものです。コンテナが再起動されるとデータは失われます。データベースやファイルアップロードなど、永続的なデータ保存が必要な場合は Kubernetes のストレージ機能を使用します。

8.2 Volume

Volume は、Pod 内のコンテナ間でデータを共有したり、一時的なデータを保存するための仕組みです。Pod のライフサイクルに Volume 連動します。

8.2.1 emptyDir

Pod が作成されると空のディレクトリが作成され、Pod が削除されると消えます。同一 Pod 内のコンテナ間でデータを共有するために使用します。

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: shared-data-pod
spec:
  containers:
    - name: writer
      image: busybox:1.36
      command: ["sh", "-c", "echo 'Hello' > /data/message.txt && sleep 3600"]
      volumeMounts:
        - name: shared-data
          mountPath: /data
    - name: reader
      image: busybox:1.36
      command: ["sh", "-c", "cat /data/message.txt && sleep 3600"]
      volumeMounts:
        - name: shared-data
          mountPath: /data
  volumes:
    - name: shared-data
      emptyDir: {}
```

8.2.2 hostPath

ノードのファイルシステム上のパスを Pod にマウントします。開発・テスト用途で使用しますが、本番環境での使用は推奨されません。

```yaml
volumes:
  - name: host-data
    hostPath:
      path: /data
      type: DirectoryOrCreate
```

8.3 PersistentVolume（PV）と PersistentVolumeClaim（PVC）

永続的なストレージを使用するには、PersistentVolume（PV）と PersistentVolumeClaim（PVC）を使用します。

- **PV**: クラスタ管理者がプロビジョニングするストレージリソース
- **PVC**: ユーザーがストレージを要求するためのリソース

8.3.1 PV の作成

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: my-pv
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  hostPath:
    path: /mnt/data
```

8.3.2 アクセスモード

モード	略称	説明
ReadWriteOnce	RWO	単一ノードからの読み書き
ReadOnlyMany	ROX	複数ノードからの読み取り専用
ReadWriteMany	RWX	複数ノードからの読み書き

8.3.3 再利用ポリシー

ポリシー	説明
Retain	PVC が削除されても PV とデータを保持する
Delete	PVC が削除されると PV とデータも削除する
Recycle	データを削除して PV を再利用する（非推奨）

8.3.4 PVC の作成

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 3Gi
```

PVC を作成すると、条件に合う PV が自動的にバインドされます。

8.3.5 PVC を Pod で使用

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pvc-pod
spec:
  containers:
    - name: app
      image: nginx:1.27
      volumeMounts:
        - name: persistent-storage
          mountPath: /usr/share/nginx/html
  volumes:
    - name: persistent-storage
      persistentVolumeClaim:
        claimName: my-pvc
```

8.4 StorageClass

StorageClass を使うと、PVC が作成されたときに動的に PV をプロビジョニングできます（動的プロビジョニング）。

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-storage
provisioner: k8s.io/minikube-hostpath
parameters:
  type: pd-ssd
reclaimPolicy: Delete
```

PVC で StorageClass を指定します。

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: dynamic-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: fast-storage
  resources:
    requests:
      storage: 10Gi
```

8.5 ストレージの確認

```bash
# PVの確認
kubectl get pv

# PVCの確認
kubectl get pvc

# StorageClassの確認
kubectl get storageclass
```

9 Namespace

9.1 Namespace とは

Namespace は、Kubernetes クラスタ内のリソースを論理的に分離するための仕組みです。同じクラスタ内で複数の環境（開発、ステージング、本番）やチームを分離する際に使用します。

9.2 デフォルトの Namespace

Kubernetes クラスタには以下の Namespace がデフォルトで存在します。

Namespace	説明
default	Namespace を指定しない場合に使用される
kube-system	Kubernetes システムコンポーネントが動作する
kube-public	全ユーザーが読み取り可能な公開リソース
kube-node-lease	ノードのハートビート情報を管理する

9.3 Namespace の操作

9.3.1 作成

```
# コマンドで作成
kubectl create namespace development

# マニフェストで作成
kubectl apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: development
  labels:
    env: dev
EOF
```

9.3.2 一覧表示

```
kubectl get namespaces
```

9.3.3 特定の Namespace 内のリソースを表示

```
# -n オプションでNamespaceを指定
kubectl get pods -n development
```

```
# すべてのNamespaceのリソースを表示
kubectl get pods --all-namespaces
# または
kubectl get pods -A
```

9.3.4 デフォルト Namespace の変更

kubectl で毎回 `-n` を指定するのが面倒な場合、デフォルトの Namespace を変更できます。

```
kubectl config set-context --current --namespace=development
```

9.4 Namespace にリソースを作成

マニフェストの `metadata.namespace` で Namespace を指定します。

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  namespace: development
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

```
kubectl apply -f nginx-deployment.yaml
```

9.5 ResourceQuota

Namespace ごとにリソースの使用量を制限できます。

```yaml
apiVersion: v1
kind: ResourceQuota
```

```yaml
metadata:
  name: dev-quota
  namespace: development
spec:
  hard:
    pods: "10"
    requests.cpu: "4"
    requests.memory: "8Gi"
    limits.cpu: "8"
    limits.memory: "16Gi"
```

9.5.1 ResourceQuota の確認

```bash
kubectl describe resourcequota dev-quota -n development
```

9.6 LimitRange

Namespace 内のコンテナに対してデフォルトのリソース制限を設定できます。

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: dev-limits
  namespace: development
spec:
  limits:
    - default:
        cpu: "500m"
        memory: "256Mi"
      defaultRequest:
        cpu: "100m"
        memory: "128Mi"
      max:
        cpu: "2"
        memory: "1Gi"
      min:
        cpu: "50m"
        memory: "64Mi"
      type: Container
```

ResourceQuota が設定されている場合、各 Pod には resources の requests と limits の指定が必須になります。LimitRange でデフォルト値を設定しておくと便利です。

9.7 Namespace の削除

```
kubectl delete namespace development
```

Namespace を削除すると、その Namespace 内のすべてのリソースも削除されます。削除前に必ず確認してください。

10 実践運用

10.1 アプリケーションのデプロイ例

実践的な例として、Web アプリケーションをデプロイする一連の流れを紹介します。

10.1.1 Deployment の作成

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
        - name: web-app
          image: nginx:1.27
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "200m"
              memory: "256Mi"
          livenessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 5
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 3
            periodSeconds: 5
```

10.1.2 Service の作成

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-app-service
spec:
  type: NodePort
  selector:
    app: web-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30080
```

10.1.3 ConfigMap の作成

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: web-app-config
data:
  APP_ENV: "production"
  APP_LOG_LEVEL: "warn"
```

10.1.4 一括適用

すべてのマニフェストを 1 つのディレクトリに配置して一括適用できます。

```bash
# ディレクトリ内のすべてのマニフェストを適用
kubectl apply -f ./manifests/

# 再帰的に適用
kubectl apply -f ./manifests/ -R
```

10.2 トラブルシューティング

10.2.1 Pod が起動しない場合

```bash
# Podの状態を確認
kubectl get pods

# 詳細情報とイベントを確認
kubectl describe pod <Pod名>
```

```
# ログを確認
kubectl logs <Pod名>

# 前回のコンテナのログを確認（再起動した場合）
kubectl logs <Pod名> --previous
```

10.2.2 よくある Pod のエラー

状態	原因	対処法
ImagePullBackOff	イメージの取得に失敗	イメージ名・タグの確認、レジストリの認証確認
CrashLoopBackOff	コンテナが繰り返しクラッシュ	ログの確認、アプリケーションの修正
Pending	スケジュールできない	ノードのリソース確認、taint の確認
OOMKilled	メモリ不足で強制終了	メモリの limits を増やす
CreateContainerConfigError	設定エラー	ConfigMap や Secret の存在確認

10.2.3 Service に接続できない場合

```
# Serviceの確認
kubectl get service <Service名>

# エンドポイントの確認（Podが紐づいているか）
kubectl get endpoints <Service名>

# ラベルの一致を確認
kubectl get pods --show-labels
```

10.3 リソースの監視

10.3.1 リソース使用量の確認

```
# ノードのリソース使用量
kubectl top nodes

# Podのリソース使用量
kubectl top pods
```

`kubectl top` を使用するには、metrics-server が必要です。Minikube では以下のコマンドで有効にできます。

```
minikube addons enable metrics-server
```

10.4 ラベルを活用した運用

10.4.1 ラベルの追加・変更

```
# ラベルを追加
kubectl label pods nginx-pod version=v1

# ラベルを上書き
kubectl label pods nginx-pod version=v2 --overwrite

# ラベルを削除
kubectl label pods nginx-pod version-
```

10.4.2 ラベルによるフィルタリング

```
# 特定のラベルを持つPodを表示
kubectl get pods -l app=nginx

# 複数条件でフィルタリング
kubectl get pods -l "app=nginx,env=production"

# ラベルの表示
kubectl get pods --show-labels
```

10.5 便利な kubectl コマンド

コマンド	説明
`kubectl get events --sort-by=.metadata.creationTimestamp`	イベントを時系列で表示
`kubectl api-resources`	利用可能なリソース種類を一覧表示
`kubectl explain <リソース>`	リソースのフィールドを確認
`kubectl diff -f <ファイル>`	適用前に差分を確認
`kubectl debug <Pod名> -it --image=busybox`	デバッグ用コンテナを起動

10.6 次のステップ

本ガイドでは Kubernetes の基礎を学びました。さらに学習を進めるために、以下のトピックを探索してみてください。

- **Ingress**: HTTP ルーティングと TLS 終端
- **Helm**: Kubernetes パッケージマネージャ
- **Kustomize**: マニフェストのカスタマイズツール
- **RBAC**: ロールベースアクセス制御
- **NetworkPolicy**: Pod 間のネットワーク制御

- **HPA（Horizontal Pod Autoscaler）**: 自動水平スケーリング
- **StatefulSet**: ステートフルアプリケーションの管理
- **Operator**: カスタムコントローラーの開発
- **GitOps**: ArgoCD、Flux を使った継続的デリバリー

10.6.1 参考リソース

- Kubernetes 公式ドキュメント: https://kubernetes.io/ja/docs/
- Kubernetes 公式チュートリアル: https://kubernetes.io/ja/docs/tutorials/
- kubectl チートシート: https://kubernetes.io/ja/docs/reference/kubectl/cheatsheet/